

Aula 04: Estrutura de um programa

Instruções elementares e estruturas de controle

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Revisitando as aulas anteriores

- ⌋ Nas aulas anteriores, discutimos uma série de conceitos básicos para elaborarmos algoritmos
 - * Discutimos a noção de algoritmo
 - * Discutimos o que são instruções elementares
 - * Discutimos o que são desvios condicionais e estruturas de repetição
- ⌋ Estamos trabalhando com um paradigma de programação que implementa diretamente um algoritmo por meio de instruções de máquina
- ⌋ Nesta aula, vamos aprender a estruturar nossos programas Python

Código-fonte de um programa

- ⌋ Ao traduzir o algoritmo para o código-fonte de um programa em uma linguagem de programação, devemos respeitar as regras de organização desta linguagem
- ⌋ Um programa Python é estruturalmente dividido em 4 partes ordenadas
 1. **Área de importações**
 2. **Área de constantes e variáveis globais**
 3. **Área de funções e classes**
 4. **Corpo do programa principal**
- ⌋ O programa é sempre lido de cima para baixo

Comentários no código-fonte

- ☾ Comentários são trechos de texto inseridos no código-fonte que pode ser usado como um apoio à documentação de um programa
- ☾ No Python podemos marcar comentários de duas maneiras
 - * **Comentário em uma linha:** Todo texto escrito após um `#` é descartado pelo interpretador, ou seja, serve apenas para programadores lendo o arquivo código-fonte
 - * **Múltiplas linhas de comentário:** todo texto escrito entre triplas de aspas duplas ou triplas de aspas simples são armazenados inicialmente em memória e depois descartados

Comentários no código-fonte

- Podemos fazer várias linhas de comentário marcando cada uma delas com `#` também

```
# Comentário em uma linha
```

```
# Comentário em múltiplas linhas  
# usando o jogo-da-velha
```

```
"""
```

```
Comentário em múltiplas linhas  
marcado com tripla de aspas duplas  
"""
```

Área de importações

- ⌋ As primeiras instruções do programa devem ser instruções de importação, ou seja, instruções que referenciam classes ou funções que existem em outros arquivos
- ⌋ Na Aula 06, veremos mais detalhes sobre a criação de módulos Python, que em essência costumam ser coleções de funções e constantes
- ⌋ Nesta primeira metade do curso usaremos, no máximo, as seguintes importações

```
from random import random, randint  
from math import pi, sqrt
```

Área de importações

⌋ **random** é uma coletânea de funções que trabalham com aleatoriedade

- * **random()** é uma função que gera um número aleatório x tal que $0 \leq x < 1$
- * **randint(a,b)** é uma função que gera um número aleatório x tal que x é inteiro e $a \leq x \leq b$, dados a e b na hora de chamar a função

⌋ **math** é uma coletânea de funções e constantes matemáticas

- * **pi** é a variável que armazena o valor de π com 15 casas decimais
- * **sqrt(x)** é uma função que calcula o resultado da raiz quadrada de um número x dado na hora de chamar a função

Exemplos

⌋ Considere os seguintes casos

- * x é um inteiro entre -4 e 30 incluindo estes números
- * y é um número real entre 2 e 3 , incluindo o 2 e excluindo o 3
- * z é a raiz quadrada de 1360

```
from random import random, randint  
from math import sqrt
```

```
x = randint(-4,30)  
y = 2 + random()      # voltaremos a isso depois  
z = sqrt(1360)
```


Área de constantes e variáveis globais

- Python é uma linguagem que não tem nenhuma sintaxe específica para declaração de constantes, ou seja, valores fixos que são armazenados na memória e que não devem sofrer alteração
- Tipicamente, em orientação a objetos, as constantes costumam ser declaradas por literais todos em letras maiúsculas
- Todavia, observe que o π é escrito como a variável **pi**

```
from math import pi
print(pi)                # 3.141592653589793
```

Área de funções e classes

- ⌋ Em seguida, em nosso arquivo de código-fonte devem ser incluídas as funções e classes com que trabalhamos
- ⌋ **Classe** é uma descrição de um tipo de dado, característica de uma linguagem orientada a objetos
 - * Nesta disciplina não trabalharemos com criação de classes, apenas criação de funções
- ⌋ A criação de funções é algo que discutiremos com mais cuidado na Aula 06
 - * Uma função é uma forma de reutilizarmos trechos de código
 - * Mas trabalharemos com algumas funções nativas, como **random**, **randint**, **sqrt**, **input**, **print** etc

Corpo do programa principal

- ⌋ Aqui ficam as instruções centrais do programa Python e podemos declarar o corpo principal de duas maneiras
 - * Escrevendo as instruções alinhadas com a margem esquerda do documento de texto do código-fonte
 - * Escrevendo as instruções dentro de um bloco condicional

```
if __name__ == "__main__":
```

- ⌋ Ao usar o condicional, garantimos que, caso importem o nosso arquivo, apenas as funções e constantes serão importadas
 - * Se o corpo principal não estiver envolvido por essa instrução, ele executa tais instruções no momento de importar o arquivo
 - * **Adotaremos em todas as aulas o corpo do programa principal dentro do bloco condicional**

☾ As instruções básicas do Python são

- * **Atribuição:** o símbolo `=` representa essa operação, sendo que antes do símbolo vem uma variável e depois do símbolo o valor ou expressão que resulta no valor a ser atrelado àquela variável
- * **Entrada:** a função **`input(mensagem)`** é a função usada para captar dados do teclado. A **`mensagem`** passada entre parênteses é um texto que é exibido ao usuário. O usuário escreve algo no terminal e aperta **`enter`** para enviar o dado
- * **Saída:** a função **`print(dados)`** é a função usada para exibir **`dados`** na tela do terminal. Esses dados podem ser um texto, uma variável ou uma sequência de textos e/ou variáveis

☾ Seja em funções ou no corpo do programa principal, cada instrução no Python deve ocupar uma única linha, mas podemos burlar essa restrição com ;

Variáveis em Python

- ⌋ Uma **variável** é um literal (cadeia de caracteres) que simboliza uma região de memória onde é armazenado um valor
- ⌋ Em qualquer linguagem de programação, os tipos de dados tem faixas de valores representáveis, o que faz com que cada tipo ocupe uma quantidade específica de bits na memória
 - * Os tipos disponíveis em Python serão um assunto para a próxima aula
 - * O tipo de uma variável em Python depende do dado armazenado nela no momento
- ⌋ A variável fica sempre antes do operador = e depois aparece o dado a ser armazenado, seja um número, um texto, o resultado de uma função, o resultado de uma expressão

Regras para nomes de variáveis

- ☾ **Deve** começar com letra maiúscula ou minúscula ou com um subscrito `_`. **Nunca deve começar com número.**
- ☾ De uma forma geral, os nomes de variáveis só podem ser compostos por subscritos, números e letras maiúsculas ou minúsculas.
- ☾ Python é *case sensitive*, i.e., considera maiúsculas e minúsculas como coisas diferentes.
- ☾ Não pode-se utilizar como parte do nome de uma variável: chaves, parênteses, '+', '-', '*', '/', '\', ',', ';', ':' e '?'.

Características da atribuição

☾ Python ainda admite a possibilidade de atribuição múltipla de valores

✱ É obrigatório que o número de variáveis e valores seja o mesmo, todos separados por vírgulas, salvo a criação de uma tupla (Conteúdo da Aula 10)

```
x = 5 ; y = 3.6 # é equivalente a x, y = 5, 3.6
```

☾ Python mantém uma tabela de referências aos objetos

✱ Um objeto pode ter um nome associado com ele, mais de um nome ou nenhum nome

✱ Cuidado que objetos mutáveis podem criar *cópias superficiais* se atribuímos uma variável a outra (discutiremos isso com calma depois)

Características da atribuição

- Em linguagens de programação com tipagem estática, a troca de valores entre variáveis é feita usando uma variável auxiliar
 - Em Python, isso pode ser evitado usando o esquema de atribuição múltipla

```
a, b = 10, 4
print(a, b)      # 10 4
a, b = b, a
print(a, b)      # 4 10
```

- Os dados de entrada sempre precisam ser armazenados em uma variável
 - O teclado sempre captura dos dados como texto
 - Podemos converter dados de textos em outros tipos (Veremos como na próxima aula)

Exemplo

⌋ A indentação em Python é obrigatória e faz parte da própria sintaxe da linguagem

✱ Este recuo determina o escopo de cada instrução

```
x = 3          # é uma constante
```

```
# Corpo do programa principal
```

```
if __name__ == "__main__":  
    y = input("Insira um valor inteiro")  
    y = int(y)  
    print("0",y,"múltiplo de 3 é",x*y)
```

```
# Os comandos precisam estar recuados para  
# pertencer a este condicional
```

Exemplo

- Quando listamos coisas dentro do **print**, o Python imprime todos os dados na mesma linha e espaçados por espaço em branco

```
x = 3          # é uma constante

# Corpo do programa principal
if __name__ == "__main__":
    y = input("Insira um valor inteiro")
    y = int(y)
    print("0",y,"múltiplo de 3 é",x*y)

# Os comandos precisam estar recuados para
# pertencer a este condicional
```

Exemplo

- ☾ A função **print** sempre pula linha após a impressão
 - * Para impedir que isso aconteça, deve-se adicionar o argumento `end=""` na lista de argumentos
 - * **Argumento** é o termo usado para qualquer valor/variável dado como entrada de uma função

```
x = 3          # é uma constante

# Corpo do programa principal
if __name__ == "__main__":
    y = input("Insira um valor inteiro")
    y = int(y)
    print("0",y,"múltiplo de 3 é",x*y)

    # Os comandos precisam estar recuados para
    # pertencer a este condicional
```

- ⌋ Além das instruções básicas, vamos ver como ficam as estruturas de controle de Python
- ⌋ Em termos de desvio condicional, temos
 - * Condicional simples, condicional composto e múltiplos caminhos
 - * A estrutura Escolha-caso é um pouco mais rebuscada do que fazemos no fluxograma ou pseudocódigo
- ⌋ Em termos de estruturas de repetição, temos
 - * Repetição indefinida $0+$ e iterador

Desvios simples e composto

- ⌋ A sintaxe do desvio simples (Se-Então) e do desvio composto (Se-Então-Senão) é praticamente uma tradução direta das instruções

```
if <condição> :                               # desvio simples
    <bloco de comandos>
```

```
if <condição> :                               # desvio composto
    <bloco de comandos 1>
else :
    <bloco de comandos 2>
```

Desvio de múltiplos caminhos

- Podemos implementar desvios de múltiplos caminhos de duas formas diferentes em Python
 - Múltiplos caminhos condicionais - usando a instrução **elif**, que é uma contração de **else if**

```
if <condição 1> :  
    <bloco de comandos 1>  
elif <condição 2> :  
    <bloco de comandos 2>  
elif <condição 3> :  
    <bloco de comandos 3>  
...  
else: # Pode ser omitido  
    <bloco de comandos caso contrário>
```

Desvio de múltiplos caminhos

- Podemos implementar desvios de múltiplos caminhos de duas formas diferentes em Python
 - Múltiplos caminhos valorados - recurso disponível a partir do Python 3.10, que utiliza uma noção de reconhecimento de padrões (usaremos de forma simplificada)

```
match <variável>:  
    case <valor 1>:  
        <bloco de comandos 1>  
    case <valor 2>:  
        <bloco de comandos 2>  
    ...  
    case _ : # Pode ser omitido  
        <bloco de comandos do caso contrário>
```

Repetição indefinida 0+ ---

⌋ Python não admite repetições indefinidas 1+, o conhecido Faça-Enquanto

- * É necessário estabelecer que, no caso da repetição indefinida 0+, o bloco de comandos deve alterar a condição que controla a repetição
- * Se o bloco de comandos não alterar a condição em hipótese alguma, então o bloco em questão ou não executa ou executa para sempre

```
while <condição> :  
    <bloco de comandos>  
else :                               # Opcional  
    <bloco de término natural>      # Opcional
```


Repetição definida iterador

- ⌋ É uma estrutura chamada de Para-Cada, que permite que uma variável assuma, a cada passagem da repetição, um valor pertencente a uma coleção iterável
 - * Se quisermos fazer contagens, precisaremos recorrer a um tipo de dados específico, os **intervalos inteiros**
 - * O iterador permite acessar diretamente o elemento independente de a coleção ser ordenada ou não

```
for <variável> in <coleção> :  
    <bloco de comandos>  
else :                                # Opcional  
    <bloco de término natural>      # Opcional
```

Else em estruturas de repetição

- Python permite definirmos um bloco de comandos para ser executado após o término usual de uma estrutura de repetição, i.e., sem ter sido interrompido por um comando **break**
 - * Este bloco é definido a partir do *else* nas repetições
 - * O comando *break* interrompe qualquer repetição
 - * Ao executar um *break*, o fluxo de execução é desviado para a próxima instrução fora do bloco do *else*, caso este esteja definido
 - * Se o **break** não é executado, o fluxo é desviado para o **else** e, depois de terminar o bloco de comandos do *else*, desviar para a próxima instrução no mesmo alinhamento do **for**

